CHARAK: Production-Readiness Review

Patient app · Doctor app · Shared package · Admin dashboard · Backend and Supabase | 10 Oct 2026
Based on reading the code and config. Analyzers, tests and builds were not run, and no code was changed.

Overall: the foundation is better than most early-stage apps. There's a real design system with tokens and generated code, shared components in `charak_core`, a payment flow that waits for the server-side webhook before treating a booking as paid, hashed OTPs with rate limits, a slot-uniqueness migration, row-level security (RLS) migrations, a non-root Docker image and a readiness check. The gaps are the ones startups usually leave until launch: security defaults, observability, CI, tests, and things that break once there are more than about 1,000 rows of data.

Severity: **High** = fix before launch · **Medium** = fix soon after · **Low** = clean-up

1. Patient app (apps/patient_app)

Good: the routing, auth redirect and 401 → logout handling are well thought out. In the payment screen, a successful Razorpay callback is treated as unconfirmed until the booking row updates, with a 90-second timeout and a "do not pay again" message. The flow is complete: directory → slot → intake (voice notes) → pay → call → rate → complaint.

Severity	Issue	Where
High	The login token (valid for 30 days) is stored in plain SharedPreferences. Health and payment apps should use flutter_secure_storage (Keychain/Keystore).	charak_core/lib/network/api_client.dart
High	No crash reporting or analytics (no Crashlytics, no Sentry). In production you won't know when the app crashes or where users drop off in the booking funnel.	everywhere
Medium	HTTP calls have no timeout. On a weak 3G connection a request can hang indefinitely with a spinner. There's also no offline or no-network state (SocketException is never caught).	api_client.dart
Medium	If the API_BASE_URL build flag is forgotten, the app points at http://10.0.2.2:8000 (emulator localhost). There are no build flavours (dev/staging/prod) to prevent that.	api_client.dart:6
Medium	Seven routes use `s.extra as String` or similar. A deep link, push-notification tap or restored app state arrives without `extra` and the app crashes. Route parameters should be in the path or query string.	router.dart
Medium	There's only one test file (push routes). Booking and payment have no widget or integration tests.	test/

Low	Unused dependencies make the app bigger: skeletonizer, flutter_animate, audioplayers, riverpod_generator/build_runner (no generated providers exist) and probably file_picker.	pubspec.yaml
Low	State management is mixed: 16 setState screens alongside Riverpod. It works, but most startups standardise on one.	screens
Low	Only 27 Semantics labels across all the Flutter code, and no localisation (intl is used only for dates). Hindi support is likely to come up soon.	all apps
Low	applicationId is com.charak.patient_app. Underscores are allowed, but the ID can never change after the first Play Store upload, so decide on it now. The Firebase config files aren't in the repo, so push notifications won't work until they're added (gitignoring them is fine).	android/app/build.gradle.kts

2. Doctor app (apps/doctor_app)

It shares `charak_core`, so it inherits every point above: token storage, timeouts, no crash reporting, the `extra` casts and the unused dependencies.

Severity	Issue
Medium	The largest screens are active_visit_screen.dart (587 lines), pricing_setup_screen.dart (435) and schedule_tab.dart (417). They mix UI, polling and API calls in one file. Pulling the logic into Riverpod notifiers would make them testable.
Medium	Doctors also get 30-day tokens, with no refresh token and no way to revoke a session. A lost phone means patient data stays accessible for up to 30 days.
Low	There are only two tests (push routes and visit navigation).
Low	Release signing silently falls back to the debug key when key.properties is missing. That's fine for development, but make CI fail a release build instead.

3. Shared package (packages/charak_core)

- **Good:** has golden tests for the component gallery, plus poller and call tests. This is the best-tested part of the Flutter code.
- **Medium:** charak_controls.dart is 1,592 lines and should be split by control type.
- **Medium:** updates use CharakPoller (polling) rather than realtime. That's acceptable for an MVP, but watch battery use and backend load as usage grows. Supabase Realtime or FCM data messages are the natural next step.

4. Admin dashboard (admin/)

Good: React Query, shadcn, generated design tokens, a mock mode (VITE_MOCK), charts and an audit log.

Severity	Issue
High	**Admin login is one shared password with a hardcoded fallback** ("charak-admin-2024" in backend/app/config.py). If the variable isn't set in production, anyone who reads this repo can log in. There are no individual admin accounts, so the audit log can't say who did what. There's no login rate limit, no 2FA, and the token lasts 30 days.
High	The admin token sits in localStorage, so it's readable by any cross-site scripting (XSS) on the page.
Medium	When the token expires, every request starts failing but the user isn't sent back to /login. The app has no 401 handling (lib/api.ts).
Medium	Nothing is paginated. /admin/bookings returns every booking with joins.
Medium	@supabase/supabase-js is installed but unused. Both shadcn (a CLI) and @base-ui/react are in runtime dependencies, alongside two toast libraries (radix toast and sonner).
Low	No tests, no ESLint or Prettier, and no route-level code splitting.

5. Backend and Supabase

Severity	Issue
High	OTP codes come from `random.choices` (otp_store.py:47), which is not a secure random generator. Use `secrets.choice`. It's a one-line fix.
High	**The dashboard figures will be wrong past 1,000 rows.** dashboard_metrics loads entire tables and counts them in Python. Supabase/PostgREST returns at most 1,000 rows by default, so the numbers will be silently wrong once you grow. Use count="exact" or a SQL view or RPC.
High	No logging (no use of the logging module anywhere), plus 13 bare `except Exception` blocks. In production, failures will be invisible. Add structured logging and Sentry.
Medium	CORS defaults to "*". Make it required in production instead of a default.
Medium	Only /auth has rate limiting. Uploads, bookings and admin login don't.
Medium	The OTP is printed when MSG91 isn't configured. Make sure startup fails in production rather than quietly printing codes to logs.
Good	Webhook signature checking is required in live mode, the stub payment endpoint is refused in production, and the readiness check and Mumbai region are right.
Low	pytest is installed in the production image. Split requirements-dev.txt.

6. Repo and process

- **High: No CI at all (no .github/workflows).** The checks listed in CLAUDE.md (analyze, test, golden tests, the token check, the admin build, pytest) only run if someone remembers to. This is the single highest-value fix.
- **Medium:** no staging environment and no Fastlane or Codemagic for store builds.
- **Medium:** migration 0005 is missing from supabase/migrations. Confirm that's intentional.
- **Low:** large binary files (the .docx requirements doc, v1-home.png) are in the repo root. Move them to docs/.

What to fix before launch, in order

1. Set up CI with GitHub Actions running the existing checks.
2. Fix admin auth: remove the default password, add per-admin accounts, 2FA or Google SSO, and rate limiting.
3. Switch OTP generation to secrets, and make startup fail if production config is missing (CORS, MSG91, admin password).
4. Add Sentry or Crashlytics to both apps and the backend, plus structured logging.
5. Move tokens to secure storage, add HTTP timeouts and offline handling, and shorten tokens with refresh tokens.
6. Fix the 1,000-row cap on dashboard metrics and add admin pagination.
7. Move route arguments out of extra so deep links don't crash.
8. Add integration tests for booking → payment → call.
9. Remove unused dependencies.